

Essential SVG for D3

Scalable vector graphics (SVG) is an XML-based markup language for describing two-dimensional graphics. It's the main graphics format used by D3 to create data-driven visualizations.

SVG shapes are created with XML tags and attributes. They are rendered by mathematical equations and can be scaled without losing resolution. Individual SVG objects can also be attached to the page's DOM tree, easily manipulated with JavaScript, styled, and transformed with CSS.

Comprehensive knowledge of SVG is important to make the most of D3, but you only really need to know the basics. It's enough to know how to create simple shapes, such as rectangles, circles, and lines, and how to change their attributes and styles. It's also useful to understand how to apply coordinate transforms, such as translation, scaling, and rotation.

This chapter provides a tutorial that explores the most important SVG features and advanced SVG DOM topics. If you already know SVG, you can safely skip it. If you are new to SVG, it is recommended that you practice creating shapes using its main tags and attributes, applying styles and transforms. After that, you will know enough SVG to start using SVG with D3.

In this chapter, we'll cover the following topics:

- Creating scalable vector graphics
- Drawing shapes, text, and paths
- Applying fills, strokes, and opacity
- Inserting images, links, and foreign objects
- Affine transforms and viewBox
- Group containers and reusable objects
- Using the SVG DOM

Creating scalable vector graphics

SVG is an open standard. SVG images can be created using a popular vector graphics editor, such as *Illustrator*, *Inkscape*, or *Canva*. SVG is also text; you can write SVG markup in any code editor, the same way you write HTML code. SVG files usually have an `.svg` extension (or `.svgz`, if compressed) and can be opened in any modern browser. You can also load them in an HTML page with the `<img>` tag. Since SVG graphics are vector-based, they are usually much smaller than bitmap images.

SVG is an XML application. Its elements are part of a *namespace* usually declared in the root element of an SVG file with the `xmlns` attribute and the identifier `http://www.w3.org/2000/svg`.

SVG can also be *embedded* inside XML and HTML documents. In HTML, adding one or more `<svg>` elements creates a graphics context. When embedded in HTML, you don't need to declare a namespace. Any valid SVG unprefixed element placed inside an `<svg>...</svg>` block will be treated as SVG:

```
<html>
<head>...</head>
<body>
  <svg> <!-- Everything in here belongs to the SVG namespace --> </svg>
</body>
</html>
```

A namespace is still required in some circumstances. For example, to create SVG elements using a DOM, instead of `createElement()`, you must use `createElementNS()`, which requires a namespace:

```
const h1 = document.createElement("h1"); // create HTML
const rect =
  document.createElementNS("http://www.w3.org/2000/svg", "rect"); // create SVG
```

Without a namespace, the element is still created but won't be rendered by the browser.

In most of this book, you never need to declare the SVG namespace, since all SVG graphics will be embedded in HTML, and all dynamic element creation will happen using D3, which doesn't require a namespace in most commands (in rare cases, it may require just the `svg` prefix).

Now that you know how SVG is created in an HTML page, let's look at its most important tags.

SVG elements

All SVG elements should be nested as descendants of the `<svg>` root element – a container that defines an initial viewport and provides a drawing canvas for shapes and other graphics.

Table 3.1 lists the main SVG elements and attributes that we will use in this book. This chapter will provide a general introduction to all of them, including code examples that you should try out.

Tag	Main attributes	Description
<svg>	height, width, viewBox	The root of an SVG. In HTML, it sets a <i>viewport</i> containing all the graphics. All elements in this table are children of the <svg> element.
<circle>	r, cx, cy	Draws a circle with radius r and the center at (cx,cy).
<rect>	x, y, width, height, rx, ry,	Draws a rectangle with specified width and height values and the top left at (x,y). Corner radii can be set with rx and ry for round-corner rectangles.
<line>	x1, y1, x2, y2	Draws a line from (x1,y1) to (x2,y2).
<ellipse>	cx, cy, rx, ry	Draws an ellipse centered at (cx,cy) with radii rx and ry.
<polyline> <polygon>	points, fill-rule	Draws an open <polyline> or closed <polygon> connecting the provided x,y coordinates. The fill-rule can be nonzero (default) or evenodd.
<path>	d	Draws an arbitrary path expressed in a compact string containing directives to draw lines and curves, set as the d (data) attribute.
<text> <tspan>	x, y, dx,dy alignment-baseline, text-anchor	Draws text with the baseline and starting point at (x,y). The baseline can be changed with the alignment-baseline property and the starting point with text-anchor. It can contain <tspan> elements, normally placed with relative dx and dy coordinates.
<image>	href, x, y	Inserts an image from a URL in the href attribute at position x,y.
<g>		A container. Styles and transforms applied to it affect its children.

Table 3.1 – Most important SVG elements

The tags in Table 3.2 are used in some of our projects but are less common in D3 visualizations. Many are effects that are easier to apply using CSS, while others duplicate functionalities that are easier to implement with D3. They are briefly introduced in this chapter.

Tag	Main attributes	Description
<defs>		Elements declared inside <defs> are not rendered unless they are referenced somewhere <i>outside</i> <defs>.
<use>	href	References any reusable element, marker, or symbol and optionally applies new attributes and styles to it.
<linearGradient> <radialGradient>	id	A gradient that can be used as a fill or stroke color for shapes. Color stops are defined by nested <stop> elements.

Tag	Main attributes	Description
<pattern>	id	Defines reusable patterns applied as a fill or stroke.
<clipPath>	id, clip-rule	Defines a custom clipping path determined by contained shape elements. Other elements apply it via the clip-path attribute.
<mask>	id	Defines an alpha mask for compositing an object into its background. Applied via the mask attribute.
<filter>	id	Defines a composite filter created by combining primitives (nested <fe...> elements). Elements apply it via the filter attribute.
<symbol>	id	A lightweight reusable graphical template object.
<marker>	id, viewBox	A reusable symbol that can be applied to parts of shapes using the marker-start, marker-mid, or marker-end attribute. Typically used to draw arrowheads and place objects on vertices.
<textPath>	id	Contains or references a path that can be used as the baseline for text. Used to make text flow on curves.
<animate>, <set>, <animateMotion>, <animateTransform>	id	Configures transitions and animations. Usually applied as children to the shape that will be animated.

Table 3.2 – Other SVG elements that may be occasionally used

Before drawing some shapes, let's understand how graphics are rendered in the SVG viewport.

The initial viewport

When SVG is embedded in HTML, it creates an initial **viewport**: a graphics context like the one created by Canvas. To create the viewport, all you need is an empty <svg/> tag somewhere in <body>. You can style it with standard (HTML) CSS and set its size using width and height attributes in pixels, as shown here:

```
<style>
  svg { border: solid 1px lightgray;
        background-color: hsla(240,100%,50%,0.2); }
</style>
<body><h2>SVG viewport</h2>
<svg width="600" height="300"></svg>
</body>
```

Figure 3.1 shows the result of the code (Containers/SVG/1-initial-viewport.html) and an overlay describing the default coordinate system used by SVG.

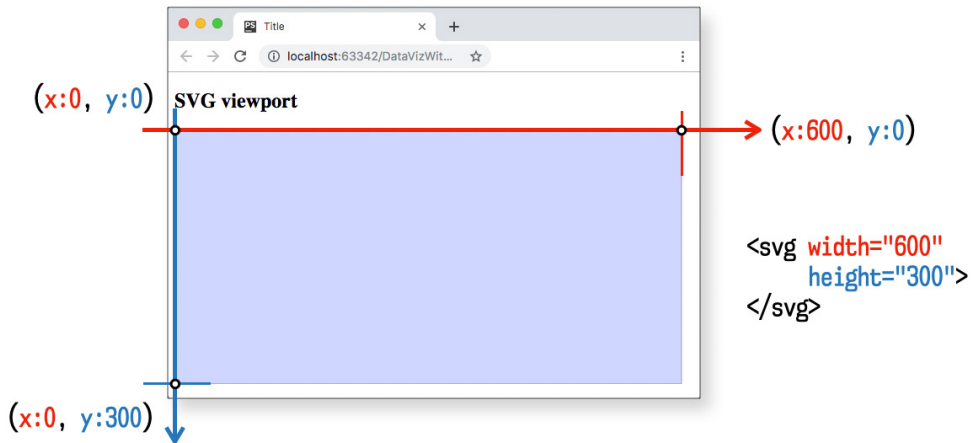


Figure 3.1 – The SVG viewport embedded in an HTML page. Code: Containers/SVG/1-initial-viewport.html

The viewport starts at the top-left corner of the drawing area, which is 600 pixels wide and 300 pixels high (from the `width` and `height` attributes). You can change the orientation, scale (units), and several other aspects of this coordinate system by configuring the `viewBox` attribute (see the *viewBox* section).

Instead of using tags, you can also create and append the SVG root element programmatically to `<body>` with pure JavaScript and the DOM API (see Containers/DOM/1-initial-viewport-dom.html):

```
const svg = document.createElementNS("http://www.w3.org/2000/svg", "svg");
svg.setAttribute("width", 400);
svg.setAttribute("height", 300);
document.getElementsByTagName("body")[0].appendChild(svg);
```

In this book, we will either include the `<svg/>` tag in `<body>` (or some other HTML container), or create it in a `<script>` block or module using D3 (see Containers/D3/1-initial-viewport.html):

```
d3.select("body").append("svg").attr("width", 400).attr("height", 300);
```

Now that we have an initial SVG viewport set up, we can start drawing shapes in it.

Drawing basic shapes

Shapes are described by XML tags (e.g., `<rect>`, `<circle>`, `<path>`, etc.). SVG graphics are created by declaring these tags in an `<svg>` block and setting required attributes (e.g., radii, points, etc.). Each tag also supports attributes that determine its position relative to the local coordinate system.

The basic shapes you will probably use most are circles, rectangles, and lines. Let's start by looking at them.

Circles

You can draw a circle with the `<circle>` element and the `r` attribute (radius). If you don't provide other attributes, you will only see the bottom-right quarter of the circle, since its default center is `0,0`.

You can place a circle by providing explicit values for its center via the `cx` and `cy` attributes. Unlike CSS properties, SVG attributes don't declare units. Values are expressed in absolute numbers, treated as *pixels* (unless transformed). The following SVG places different circles in a 200x200-pixel SVG canvas:

```
<svg width="200" height="200">
  <circle r="25" />
  <circle cx="40" cy="40" r="20" />
  <circle cx="100" cy="100" r="50" />
  <circle cx="200" cy="200" r="75" />
</svg>
```

The result of the preceding code is shown in *Figure 3.2*:



Figure 3.2 – Drawing circles in the SVG coordinate system. Code: `Shapes/SVG/1-shapes.html`

You can obtain an equivalent result using the DOM or D3. The following code creates the same elements and attributes with D3 (see `Shapes/D3/1-shapes.html`):

```
const svg = d3.select("body").append("svg").attr("width", 200).attr("height", 200);

svg.append("circle").attr("r", 25);
svg.append("circle").attr("cx", 40).attr("cy", 40).attr("r", 20);
svg.append("circle").attr("cx", 100).attr("cy", 100).attr("r", 50);
svg.append("circle").attr("cx", 200).attr("cy", 200).attr("r", 75);
```

Of course, the preceding code could be much shorter if we had used D3's data-binding features or even simple loops, but code duplication was maintained so you can compare it to the XML version.

SVG circles are used in D3 to create scatterplots and bubble charts.

Rectangles

A rectangle is created with the `<rect>` element and requires at least width and height. The `x` and `y` attributes determine its top-left corner. The following code draws a cross with two rectangles:

```
<svg width="200" height="200">
  <rect x="95" y="10" width="10" height="180" />
  <rect x="10" y="95" width="180" height="10" />
</svg>
```

Figure 3.3 shows the result:

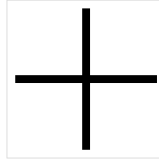


Figure 3.3 – Drawing rectangles in the SVG coordinate system. Code: Shapes/SVG/1-shapes.html

You can create rectangles with rounded corners using the `rx` and `ry` attributes, which determine the corner radius. In D3, rectangles are used to create classical bar charts.

Lines

Lines are created with `<line>` and require at least four attributes. The coordinates of the endpoints are provided by the `x1`, `y1`, `x2`, and `y2` attributes. Lines have a default width of 1 pixel, but no default stroke color; they will be invisible unless you set one. The simplest way to do this is via the `stroke` attribute.

The following code draws an X in the middle of the viewport using two lines:

```
<svg width="200" height="200">
  <line x1="10" y1="10" x2="190" y2="190" stroke="black"/>
  <line x1="10" y1="190" x2="190" y2="10" stroke="black"/>
</svg>
```

Figure 3.4 shows the result of the preceding code:

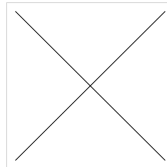


Figure 3.4 – Drawing lines in the SVG coordinate system. Code: Shapes/SVG/1-shapes.html

The `stroke` attribute can be applied to any shape or path. You will learn more about them in the *Applying styles* section.

Most lines in D3 are created using paths, which are more flexible, but sometimes using `<line>` is simpler. The same is true about ellipses and polygons.

Ellipses

Ellipses have a center at `cx` and `cy` like circles, but also two radii: the semi-major and semi-minor axes, set with `rx` and `ry`. This code creates a simple ellipse with a horizontal semi-major axis:

```
<svg width="200" height="200">
  <ellipse cx="100" cy="100" rx="90" ry="45"/>
</svg>
```

The result is shown in *Figure 3.5*:

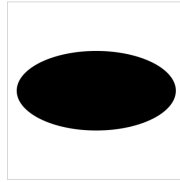


Figure 3.5 – A simple ellipse created with SVG. Code: [Shapes/SVG/1-shapes.html](#)

Ellipses are either tall or wide. If you need an ellipse in another position, you can rotate or shear it using transforms. See [Shapes/SVG/8-ellipses.html](#) for some examples of transformed ellipses.

Polygons and polylines

Polygons (`<polygon>`) and polylines (`<polyline>`) render closed or open shapes using straight lines specified by a list of space-separated vertex *x,y* coordinates passed in the `points` attribute. The only difference between them is that polygons close the shape automatically, while in polylines, you must repeat the initial coordinates to achieve the same result. A `fill-rule` sets the winding order and determines whether a hole will be drawn inside the shape when an outline crosses itself.

The following code creates two identical polygons. To shift the second polygon to the right, we added 200 to each *x* coordinate. Since it was created with `<polyline>`, the first coordinates were repeated at the end to close the shape. It also declares a `fill-rule` that will reveal a hole:

```
<svg width="600" height="300">
  <polygon points="150,150 50,150 150,50 200,200 20,200 32,20" fill="blue"/>
  <polyline points="350,150 250,150 350,50 400,200 220,200 232,20 350,150"
    fill="red" fill-rule="evenodd"/>
</svg>
```

The preceding code was used to draw the shapes shown in *Figure 3.6*.

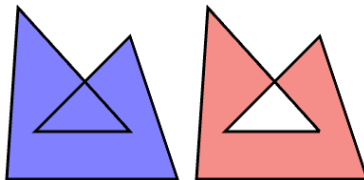


Figure 3.6 – Polygons and polylines created with SVG. Code: [Shapes/SVG/9-polygon.html](#)

Instead of adding an offset to each coordinate, you can move a shape using transforms. You can even reuse the same shape. See examples using these features in [Shapes/SVG/9-polygon.html](#).

Applying styles

Shapes can be styled with *XML attributes* (such as `stroke`) or *CSS properties* with the same name, which can be applied inline (via the `style` attribute) or with a CSS stylesheet.

Fills and strokes

Shapes have default black *fill* colors and transparent *stroke* borders, unless you assign different color strings via the fill and stroke SVG attributes (or CSS properties).

The following SVG contains three straight lines. They would be invisible if the stroke attribute (or CSS property) wasn't present, since the default stroke color is none. Besides the stroke color, you can also define the line's width with stroke-width, which has the default value of 1:

```
<svg width="600" height="200">
  <line x2="400" stroke="red" stroke-width="5"/>
  <line y2="150" stroke="blue" stroke-width="5"/>
  <line x2="200" y2="150" stroke="black"/>
</svg>
```

By default, missing coordinate attributes are zero. In the preceding code, since x1 and y1 are zero in all lines, they all start at the top left. *Figure 3.7* shows the result.



Figure 3.7 – Drawing lines with different stroke colors and stroke widths. Code: Shapes/SVG/2-line-stroke.html

You can control other stroke attributes (see Shapes/SVG/3-rect-stroke.html and *Table 3.3*). To outline a shape, set its fill property to none or fill-opacity to 0, since shapes have opaque black fills by default.

Attribute	Description
fill	The fill color of the shape. Default: '#000' (black).
stroke	The stroke color of the shape. Default: 'none'.
opacity	The opacity of the shape. A value from 0 (transparent) to 1 (opaque). Default: 1.
fill-opacity	The opacity of the fill color. This is multiplied by the opacity value.
stroke-opacity	The opacity of the stroke color. This is multiplied by the opacity value.
stroke-width	Stroke width, in units (pixels, in a default view box). Default: 1.
stroke-linecap	Shape to be used at the ends of open paths or dashes. Can be butt, round, or square. Default is butt.
stroke-dasharray	Dashing pattern. A sequence of integers alternating a solid line and a space. For example, the pattern 5 5 will produce 5-unit dashes followed by 5-unit spaces.

Table 3.3 – Attributes related to fills and strokes

All XML attributes can also be applied as CSS properties with the same names.

CSS properties

Style rules can also be set via CSS properties with the same names as the XML attributes. CSS, however, has precedence over XML: a style with the same name as an existing attribute will override the attribute. Adding the following style declaration in the `<style>` block of your page will change the width of all lines to 20 pixels, ignoring the existing `stroke-width` attributes set in each element:

```
line { stroke-width: 20px; }
```

This result is shown in *Figure 3.8*. The lines at the edges *seem* thinner because half of their width is outside the viewport:



Figure 3.8 – Overriding XML style attributes with CSS. Code: `Shapes/SVG/4-line-css.html`

Inline styling with the `style` CSS attribute, due to the cascade rule, overrides any styles previously applied via a stylesheet. You can also use the CSS class and id attributes. For example, consider the following stylesheet with CSS class declarations:

```
.reds { fill: red; }
.semitr { fill-opacity: 0.5; }
```

Since the following rectangles each belong to one or more of these classes, they will inherit the style properties declared for each class:

```
<svg width="600" height="200">
  <rect x="50" y="50" width="90" height="90" class="semitr"/>
  <rect x="200" y="50" width="175" height="100" rx="40" ry="40" class="reds semitr"/>
  <rect x="450" y="25" width="100" height="150" class="reds"/>
</svg>
```

One class applies the red fill, and the other applies 50% opacity. The rectangle in the middle belongs to both classes, so it's both red and semi-transparent, as shown in *Figure 3.9*:



Figure 3.9 – Applying CSS classes to SVG rectangles. Code: `Shapes/SVG/6-rect-css-class.html`

Although HTML and SVG share many CSS properties and can share the same stylesheet, some properties, such as `fill` and `stroke`, only exist in SVG. In HTML, `background-color` and `border-color` are used, which don't work for SVG elements. The only exception is the `<svg>` element itself, which is filled and stroked with `border-color` and `background-color`. But for any elements declared *inside* the `<svg>` element, such as `<rect>`, you need to use `stroke` and `fill`.

SVG shapes can be stroked and filled with standard CSS color names (such as red, blue, or navy), hex codes (such as #ff0000 or #807be5), or color functions (rgb(100%, 50%, 25%), rgba(120,255,80,0.5), hsl(240,0.5,1), etc.) You can also fill and stroke with patterns and gradients.

Opacity

In SVG, overlapping objects follow code order; elements declared later appear on top. The CSS z-index property doesn't apply to SVG. To bring an object to the front, you must modify the DOM tree.

You can see through overlapping objects by adjusting their opacity with fill-opacity, stroke-opacity, or opacity (value between 0 and 1). An alternative is to set the alpha component in rgba or hsla CSS color functions. For example, the following CSS rule applies a 50% red fill to all circles:

```
circle { fill: rgba(255,0,0,0.5); }
```

In the following example, the first two thin vertical rectangles appear behind a wide horizontal rectangle, and the last two appear in front of it:

```
<rect x="300" y="50" height="150" width="25"/>           <!-- Rectangle 1 -->
<rect x="400" y="50" height="150" width="25" fill-opacity=".5"/> <!-- Rectangle 3 -->
<rect x="250" y="100" height="50" width="300"
    fill="red" stroke="orange" stroke-width="10" stroke-opacity=".5" />
<rect x="350" y="50" height="150" width="25"/>           <!-- Rectangle 2 -->
<rect x="450" y="50" height="150" width="25" fill-opacity=".5"/> <!-- Rectangle 4 -->
```

Figure 3.10 shows the result of the preceding code. Note that the first and third thin rectangles from the left were declared before the red rectangle and the second and fourth rectangles.

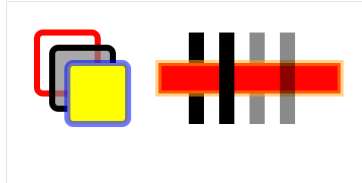


Figure 3.10 – Rectangles with different fills, strokes, and transparency. Code: Shapes/SVG/7-rect-opacity.html

Now that you know how to create basic shapes, you can use them as primitives and combine them to create complex drawings. In the next section, you will learn another way to create shapes.

Drawing paths

You can draw shapes that mix lines, curves, and arcs using a compact drawing language in the d attribute of the <path> element. It can be used to create arbitrary paths:

```
<path d="M100,200 C200,50 300,100 300,200 L400,250 500,100"
    fill="yellow" stroke="red" stroke-width="4"/>
```

The numbers and letters in the `d` attribute are directives that *move* (M) to the x,y position (100,200), draw a *Bezier curve* (C) with two control points at (200,50) and (300,100), ending in (300,200), and end with a *line* (L) to (400,250) and then to (500,100). In *Figure 3.11*, dots were added to show the control points:

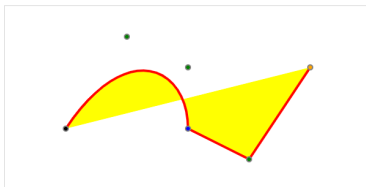


Figure 3.11 – A curve described by a `<path>` element. Code: `Paths/SVG/1-path-line.html`

Path strings are typically generated by software, but understanding them can be useful for debugging. Directives in *Table 3.4* are followed by one or more values (separated by commas or spaces), which are *absolute* coordinates, for uppercase commands, or *relative* coordinates, for lowercase commands.

Directive	Coordinates	Description
M, m	$x\ y$	Move to. Moves to an x,y position.
L, l	$x\ y$	Line to. Draws a line to an x,y position.
V, v	y	Vertical line. Draws a vertical line to a y position.
H, h	x	Horizontal line. Draws a horizontal line to an x position.
C, c	$x1\ y1\ x2\ y2\ x3\ y3$	Bezier. Draws a Bezier curve from the current x,y position to $x3,y3$, using the control points $x1,y1$ and $x2,y2$, and ending in $x3,y3$.
S, s	$x1\ y1\ x2\ y2$	Smooth Bezier. Must follow a C/c command. Draws a Bezier using the second control point from the previous curve to infer the first control point, $x1,y1$ as the second, and ending in $x2,y2$. See an example in <code>Paths/SVG/2-path-curve.html</code> .
Q, q	$x1\ y1\ x2\ y2$	Quadratic Bezier. Draws a Bezier with one control point $x1,y1$, ending in $x2,y2$.
T, t	$x1\ y1$	Smooth Quadratic Bezier. Must follow a Q/q command. Draws a Bezier using the control point from the previous curve to infer the control point, ending in $x1,y1$.
A, a	$rx\ ry\ rotx\ la\ sw\ x\ y$	Arc. Draws an arc with horizontal and vertical radii (rx,ry); the x -axis rotation angle in degrees $rotx$; two flags, la (large-arc) and sw (sweep), which select one of the four possible arcs; and the final x,y position. See <code>Paths/SVG/3-path-arc.html</code> .
Z, z	None	End. Closes a curve.

Table 3.4 – Shape directives for the `d` attribute in `<path>`

You will find some simple path examples containing lines, curves, and arcs in the `Paths/` folder.

Inserting text

Unlike HTML, you can't simply insert text in an SVG element. You need to create text objects with the `<text>` element and add the text to it. Text is positioned using `x` and `y` attributes, where `y` is the *baseline*. If `y` is 0 (or not present), only the parts of the text that extend below the baseline are visible.

The following example places both text and a rectangle in the same position:

```
<rect x="0" y="0" height="36" width="200"/>
<text font-size="36" x="0" y="0" fill="lightgray">ghijklmnop</text>
```

Figure 3.12 shows the result of different alignment-baseline values applied to the `<text>` element, showing the effects of the default "baseline" (left), "middle" (middle), and "hanging" (right).



Figure 3.12 – Drawing text in SVG and the baseline. Code: Text/SVG/1-text.html

Text can be aligned horizontally with `text-anchor` (Figure 3.13). When rotating text relative to its position, both values used in `text-anchor` and `alignment-baseline` will affect the result.

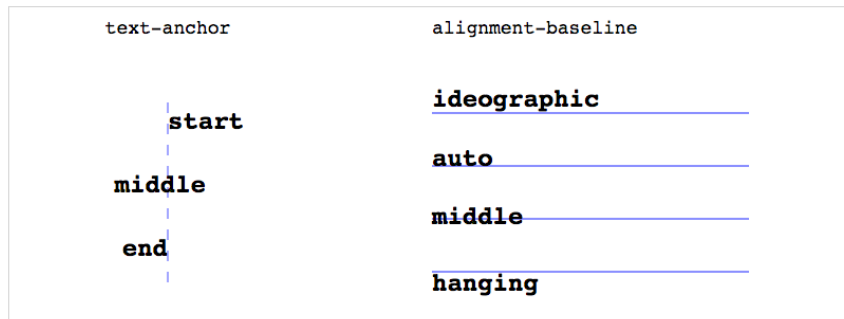


Figure 3.13 – Properties for aligning text. Code: Text/SVG/2-text-align.html

Besides the `x` and `y` attributes, the text's relative position can be controlled with `dx` and `dy`. This makes it easier to place multiline text (but still not as easy as in HTML). In these cases, you can use `<text>` as a container for one or more `<tspan>` elements (see Text/SVG/3-tspan.html).

Just like any other object, a text object can be moved, rotated, scaled, or skewed. It can also be rendered along a path with `<textPath>`. The following code declares a path in a `<defs>` block (making it invisible), which is then referenced via its ID by `<textPath>`, which contains the text to be displayed:

```
<defs>
  <path id="road" ... d="..." />
</defs>
<text text-anchor="middle">
```

```
<textPath startOffset="50%" href="#road">
  This text is centered and uses the invisible path defined above as a baseline.
</textPath>
</text>
```

The result is shown in *Figure 3.14*.

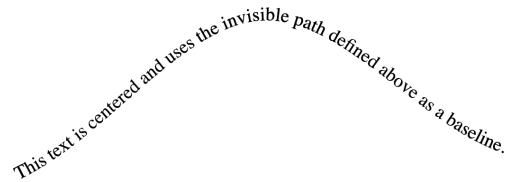


Figure 3.14 – Using `<textPath>` to draw text over a curved line. Code: Text/SVG/4-textPath.html

The `startOffset` attribute determines where the text starts, relative to the path's total length. Other attributes can also be used to distribute text along the path (see links to the documentation in *Chapter 20*).

Inserting images, links, and foreign objects

The `<image>` element allows a raster image to be imported into an SVG. It requires the same attributes as the `<rect>` element plus an `href` attribute containing the URL of the image. The following code (Links/SVG/1-image.html) loads an image and displays it in the viewport:

```
<image x="25" y="25" height="350" width="350" href="../../data/images/pluto.jpg"/>
```

The `<a>` element creates a link in SVG. It works like the HTML `<a>` and requires an `href` attribute with the URL of the linked resource. The element's body should contain the clickable elements that trigger the link.

The following code will download an image file when the user clicks on the text and place it in a browser window or frame named 'result':

```
<a href="../../data/flags/CHN.jpg" target="result"><text>...</text></a>
```

`<foreignObject>` is an extension point to include code from HTML or other XML applications. See Links/SVG/2-foreign-object.html for an example that embeds an `<iframe>` in SVG.

Before exploring more elements, let's learn how to move, scale, and rotate elements with transforms. This is a rather complex topic, but with simple examples, we can make the learning curve smoother.

Affine transforms

The initial viewport establishes a default coordinate system used by all child elements, where each unit in any direction corresponds to one pixel and coordinates start at the top-left corner. This coordinate system can be modified in two ways: setting the `viewBox` attribute or applying affine transforms.

The transform XML attribute (also a CSS property) applies affine transforms (matrix multiplication) to any shape or group of shapes. The operation is set up by assigning values to the six components that form the first two rows of the transformation matrix, which is the identity matrix by default (no transformation). This can be done with the `matrix()` function (Figure 3.15).

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \begin{bmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{bmatrix} \quad \text{matrix}(a,b,c,d,e,f)$$

Identity matrix Transformation matrix components

Figure 3.15 – The identity matrix and the matrix used for affine transforms. The a, c, b, and d components represent linear transformations (rotation, scaling, shearing). The e and f components represent translation

Note that the parameters are ordered by column, not by row. The default component values, which correspond to no transformation, are a,d = 1, and b,c,e,f = 0. For example, if you apply the following transform to a circle, nothing happens, since no components were changed:

```
<circle cx="25" cy="25" r="50" transform="matrix(1,0,0,1,0,0)"/>
```

The following transform changes the last two matrix components (the third column), which performs a *translation* operation. This moves the circle 75 pixels left and 50 pixels down:

```
<circle cx="25" cy="25" r="50" transform="matrix(1,0,0,1,75,50)"/>
```

This operation effectively changes the coordinate system used by the circle. The position of its center (cx and cy) is now relative to the new *local coordinate system* created by the transform. The circle's absolute position (relative to its container) has changed to (100,75), which is the sum of its center coordinates and corresponding translation parameters (Figure 3.16).

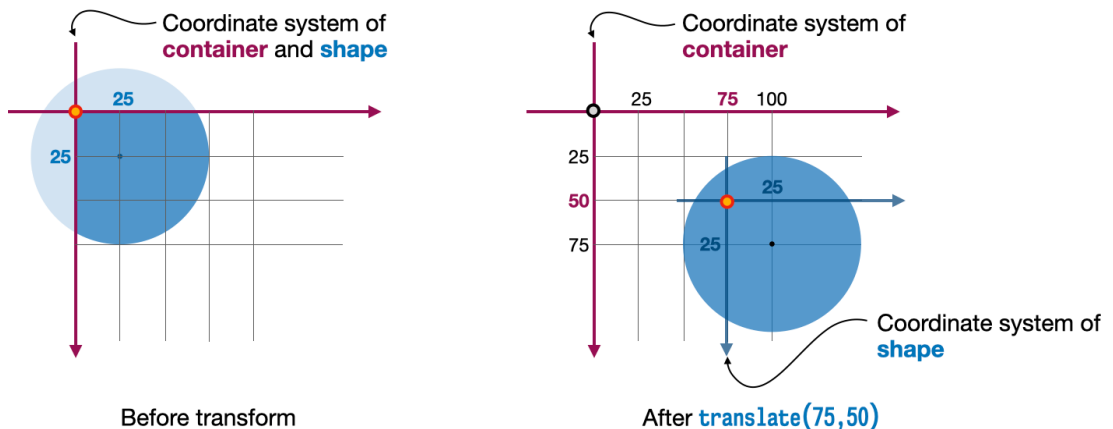


Figure 3.16 – Before and after applying a coordinate transform on a circle. Code: <Transforms/SVG/1-translate.html>

Of course, in this simple example, you could obtain the same visual result by simply changing the circle's `cx` and `cy` attributes, but transforms are much more powerful, as we will see.

You can use `matrix()` to perform any kind of transformation, but it's usually simpler to call specific operations that just change the relevant components, especially when it involves angles, such as shearing or rotation. The same transformation as shown previously could be performed with the following:

```
<circle cx="25" cy="25" r="50" transform="translate(75,50)"/>
```

Table 3.5 lists and describes all the operations you can perform using the `transform` attribute.

Operation	Description
<code>translate(x,y)</code>	Moves to x,y . Same as <code>matrix(1,0,0,1,x,y)</code> . Translation along the x axis requires a single argument: <code>translate(x)</code> is the same as <code>translate(x,0)</code> .
<code>scale(h,v)</code>	Scales by a factor of h (x axis) and v (y axis). Same as <code>matrix(h,0,0,v,0,0)</code> . To scale the same factor on both axes, provide a single argument: <code>scale(z)</code> is the same as <code>scale(z,z)</code> . Negative values flip about the opposite axis.
<code>rotate(a,x,y)</code>	Rotates by a degrees at x,y . Equivalent to <code>matrix(cos a, -sin a, sin a, cos a, x, y)</code> . The last two arguments are optional: <code>rotate(a)</code> is <code>rotate(a,0,0)</code> . The <code>transform-origin</code> attribute (or CSS property) can also be used to set the center of rotation.
<code>skewX(a)</code>	Shears horizontally by a degrees. Same as <code>matrix(1,0,tan a,1,0,0)</code> .
<code>skewY(a)</code>	Shears vertically by a degrees. Same as <code>matrix(1,tan a,0,1,0,0)</code> .
<code>matrix(a,b,c,d,e,f)</code>	All cumulative transforms can be applied with a single <code>matrix()</code> command. <i>Figure 3.17</i> shows how the preceding basic transforms are related to <code>matrix()</code> .

Table 3.5 – SVG coordinate transforms (used in the `transform` attribute)

Although it's usually more convenient to apply simple transforms using `translate()`, `scale()`, and `rotate()`, it's useful to know how these functions are related to the `matrix()` syntax (*Figure 3.17*).

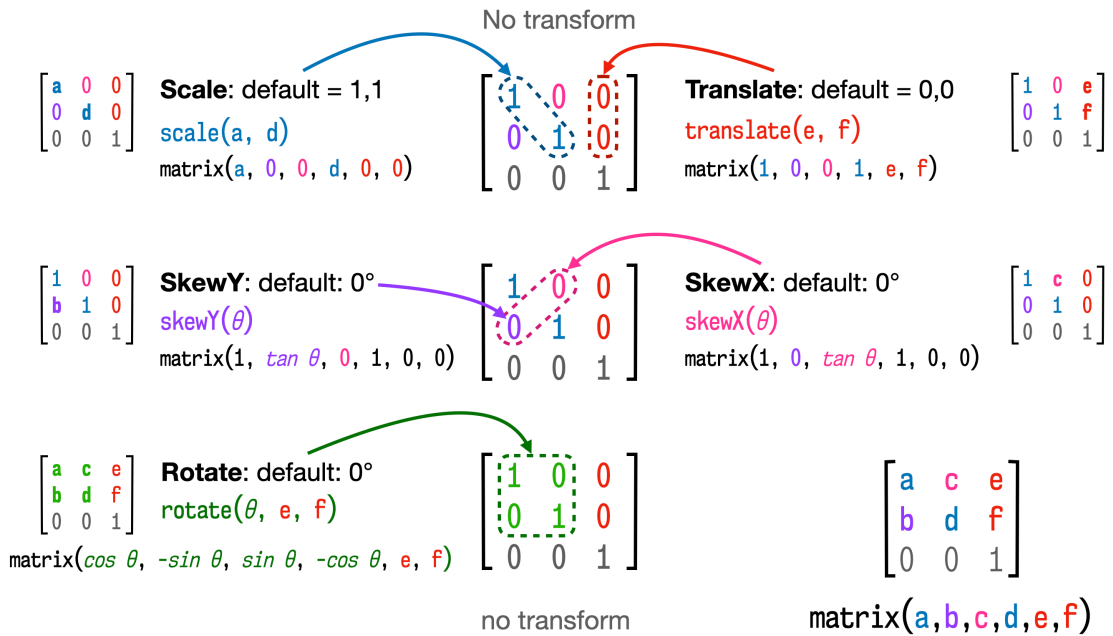


Figure 3.17 – Affine transformations, the functions used to apply them in SVG, and affected matrix components

 **Quick tip:** Need to see a high-resolution version of this image? Open this book in the next-gen Packt Reader or view it in the PDF/ePub copy.

 **The next-gen Packt Reader** and a **free PDF/ePub copy** of this book are included with your purchase. Scan the QR code OR go to packtpub.com/unlock, then use the search bar to find this book by name. Double-check the edition shown to make sure you get the right one.



The **current transformation matrix (CTM)** contains the cumulative transforms applied to an object. Using the DOM, you can obtain an object's CTM and extract the six components required by the `matrix()` operation. We will explore the CTM in the *Using the DOM API* section.

Cumulative transforms

A single transform can perform a sequence of cumulative operations. For example, the following code will translate and then scale the circle, displaying it in half its original size:

```
<circle cx="25" cy="25" r="50" transform="translate(75,50) scale(0.5)"/>
```

Note that each operation runs in a coordinate system created by the preceding operation. This means that if you scale first, the translation will occur in the new scaled coordinate system where one unit corresponds to a half-pixel, placing the circle in different absolute coordinates (*Figure 3.18*).

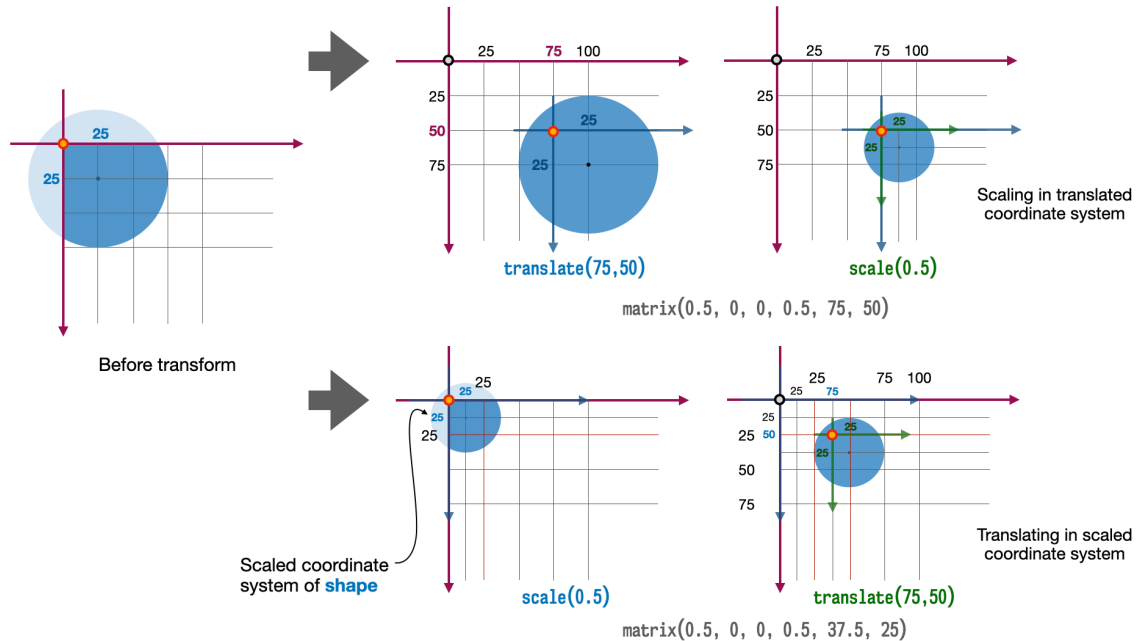


Figure 3.18 – The order of transform operations affects the results. Code: [Transforms/SVG/2-cumulative.html](#)

Note that the matrices, which represent the cumulative transforms, are different. To achieve the same effect, you would need to double the *x* and *y* parameters for the `translate()` operation:

```
<circle cx="25" cy="25" r="50" transform="scale(0.5) translate(150,100)"/>
```

You can compare these results in [Transforms/SVG/2-cumulative.html](#).

Transforms in CSS

The same transforms applied with the `transform` XML attribute can be applied in CSS, but since CSS doesn't use default units, you must append the unit after the values (e.g., 'px', 'deg', etc.)

For example, consider the following transform using the XML attribute:

```
<rect ... transform="rotate(-30,50,25) translate(10,15)" />
```

In CSS, you can apply it in a stylesheet (see [Transforms/SVG/3-transform-css.html](#)), but it must use units:

```
rect {
  transform-origin: 50px 25px;
  transform: rotate(-30deg) translate(10px,15px);
}
```

There are also some differences. XML attributes only support the functions listed in *Table 3.5* (CSS supports more), and the `rotate()` function in CSS takes a single argument (to set the center, you need to set the `transform-origin` property, as shown previously).

Flipping an object

Calling `scale()` with negative numbers will reflect the object's coordinate system about the origin or an axis. The following transform flips the polygon about the origin without changing its size:

```
<polygon points="0,0 100,5 75,25 100,45 0,50" transform="scale(-1)"/>
```

Calling `scale()` with two arguments applies scaling to the x and y axes separately. If the first argument is positive and the second is negative, the transform will flip the object vertically, about the x axis. Inverting the signs flips the object horizontally, about the y axis:

```
<polygon points="0,0 100,5 75,25 100,45 0,50" transform="scale(-1,1)" />
```

If your origin is located at the default top left, all these transforms move the object offscreen. To flip an object about its center, you must change its transformation origin. The best way to do this is to set `transform-origin` with the object's center coordinates:

```
<polygon points="..." transform-origin="50 25" transform="scale(-1,1)" />
```

These transforms are shown in *Figure 3.19*. The circle indicates the origin used for each transform.

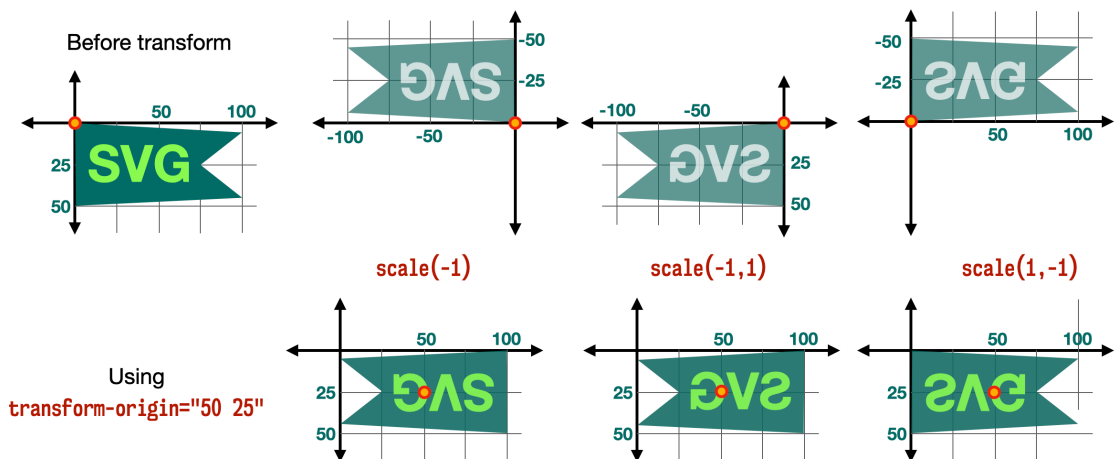


Figure 3.19 – Using `scale()` to flip an object. Code: [Transforms/SVG/4-scale-flip.html](#)

The `transform-origin` attribute (also a CSS property) is particularly useful in operations that involve angles, such as rotating and shearing.

Rotating and shearing

A `matrix()` operation for rotating or shearing requires computing sines and cosines, or tangents of angles in radians. It's more convenient to use `rotate()`, `skewX()`, and `skewY()`, which take angles in degrees.

The rotation origin is provided by the shape's parent container, the previous transform, or the `transform-origin` property. Setting the correct origin is critical to avoid rotating a shape offscreen. The following polygon, for example, is rotated -45 degrees. It would vanish if rotated -90 degrees:

```
<polygon points="0,0 100,5 75,25 100,45 0,50" transform="rotate(-45)"/>
```

To rotate it around its center, either use `rotate()` with three arguments or use `transform-origin`:

```
<polygon points="..." transform-origin="50 25" transform="rotate(-45)"/>
```

Figure 3.20 shows the effect of rotating about the origin and about the center of the shape.

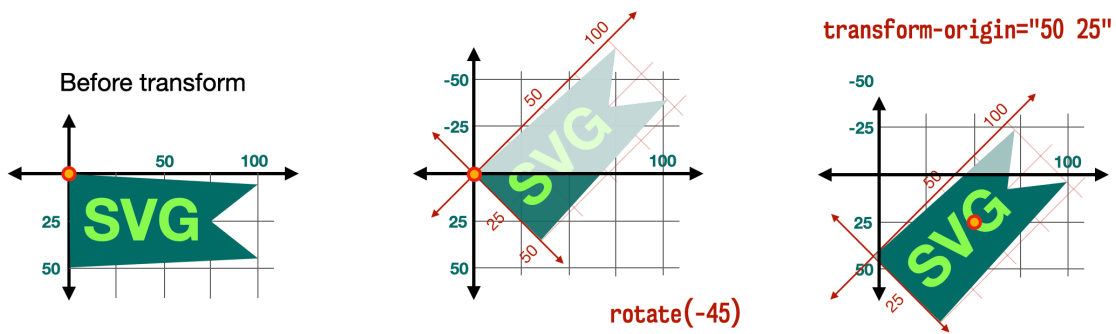


Figure 3.20 – Using `rotate()` to rotate an object 45 degrees counterclockwise.

Code: <Transforms/SVG/5-rotate-skew.html>

You will find more examples in the `Transform/` folder, which also contains examples using shearing.

Group containers

Groups are lightweight containers that wrap SVG elements in a `<g>` block. They don't have height, width, x, or y attributes. To move a group, you need to apply a transform:

```
<svg width="900" height="300">
  <g>
    <circle cx="150" cy="150" r="120" fill="none" stroke="currentColor"/>
    <rect x="30" y="140" height="20" width="240" fill="currentColor"/>
    <rect x="140" y="30" width="20" height="240" fill="currentColor"/>
  </g>
  <g color="red" transform="translate(300,0)">
    <circle cx="150" cy="150" r="120" fill="none" stroke="currentColor"/>
```

```

    <rect x="30" y="140" height="20" width="240" fill="currentColor"/>
    <rect x="140" y="30" width="20" height="240" fill="currentColor"/>
  </g>
</svg>

```

Groups can't be filled or stroked but a `<g>` with stroke and fill attributes applies these styles to descendants that don't define these attributes.

If the elements in a group set their fill or stroke value to `currentColor`, you can set the color attribute in a group to apply the same color to fill and stroke its descendants, as shown previously.

The preceding code produces the image shown in *Figure 3.21*.

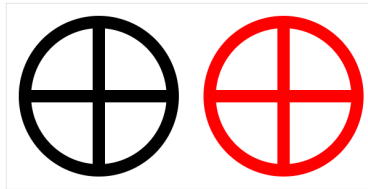


Figure 3.21 – Applying a color to strokes and fills using `currentColor`. Code: Containers/SVG/2-groups.html

Applying a transform to a group creates a new coordinate system. The attributes of the shapes in the second group are *relative* to this new system. The shapes in both `<g>` blocks have the same coordinates.

Reusable objects

The `<defs>` element stores definitions for reusable structures. Shapes or groups declared in it are *not* rendered. They can, however, be referenced outside the `<defs>` block and be used multiple times.

Instead of duplicating code to draw a new symbol, here we declare the same group inside a `<defs>` block, with an `id="earth"` attribute. It won't appear on the screen, but it can be reused:

```

<defs>
  <g id="earth">
    <circle cx="150" cy="150" r="120" fill="none" stroke="currentColor"/>
    <rect x="30" y="140" height="20" width="240" fill="currentColor"/>
    <rect y="30" x="140" width="20" height="240" fill="currentColor"/>
  </g>
</defs>

```

The `<use/>` element reuses any object that has an `id`. In the following example, it reused the `#earth` group three times, applying different colors and transforms in each case:

```

<use href="#earth" />
<use href="#earth" color="red" transform="scale(0.5) translate(600,0)" />
<use href="#earth" color="blue" transform="translate(450,0) rotate(45,150,150)" />

```

The result is shown in *Figure 3.22*.

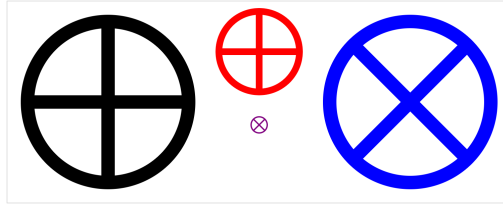


Figure 3.22 – Reusing symbols declared in a `<defs>` block with `<use>`. Code: Containers/SVG/3-defs-use.html

Objects can also be reused using `<svg>` in `<defs>` (see Containers/SVG/5-svg-reuse.html). Unlike `<g>`, an `<svg>` can have width and height and can set a new coordinate system with the `viewBox` attribute.

viewBox

The optional `viewBox` attribute, supported by `<svg>` and a few other containers, sets a rectangular viewport with local units. It contains four numbers separated by spaces. The top-left corner, which defaults to `0 0`, and *width* and *height*, which default to the declared or default pixel height and width of the container. This attribute can move the viewport to a different position and change its local units.

For example, a `viewBox` declared as `0 0 400 300` in a 400x300-pixel SVG changes nothing:

```
<svg width="400" height="300" viewBox="0 0 400 300">
  <rect width="350" height="250"/>
  <circle r="100"/>
  <line y2="90"/>
  <line x2="90"/>
</svg>
```

This SVG draws a rectangle, a circle, and two lines at 0,0. Only the bottom-right quadrant is visible since the origin coincides with the top-left corner of the viewport (*Figure 3.23*).

The circle has a radius of 100 units, rendered as 100 pixels. Since a circle's default origin is its center, only the bottom-right quarter of the circle is visible. The two lines start at the origin (0,0), which is also where the center of the circle is placed. One moves 90 pixels down, while the other moves 90 pixels right. Only half of their stroke is visible. A rectangle's default origin is its top-left corner. Since the rectangle is smaller than the viewport, it's fully visible. Only its top and left strokes are thinner, since half their width is offscreen.

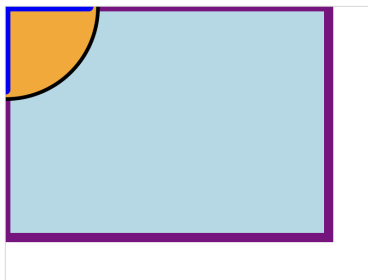


Figure 3.23 – A circle at the origin in a default viewport. Code: Containers/SVG/6-viewBox-default.html

You can use `viewBox` to move the top-left corner. For example, `viewBox="-200 -150 400 300"` will place the origin at the *center* of the viewport, by moving the top-left corner halfway up and left. Now, with the origin at the center, you can see the entire circle and full stroke widths (Figure 3.24).

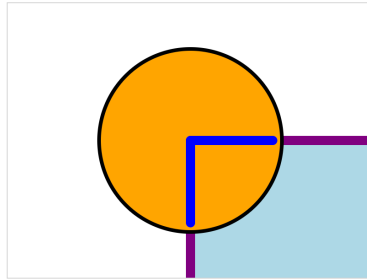


Figure 3.24 – Moving the origin to the center of the viewport. Code: Containers/SVG/7-viewBox-center.html

`viewBox` can also be used to scale the coordinate system. With `viewBox="-400 -300 800 600"`, it now contains twice as many units per pixels, while still at the center. This will display all the objects at half their original size, since the 400-pixel-wide SVG now fits 800 units. This means that a circle with a radius of 100 units will be rendered with a 50-pixel radius (Figure 3.25).

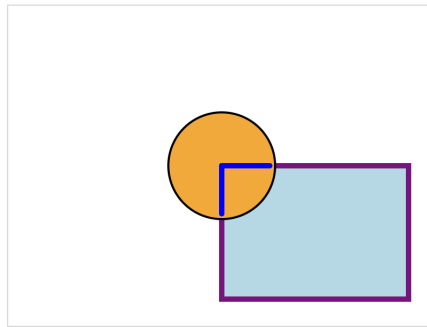


Figure 3.25 – In this viewport, each pixel corresponds to two units. Code: Containers/SVG/8-viewBox-scaled.html

You can scale the contents of the container by changing its width and height attributes to fit the viewport wherever you want. Using percentages for height and width fits it in its parent container (e.g., a `<div>`), allowing for responsive graphics. If width and height are not proportional to `viewBox` dimensions, the contents are scaled, preserving the aspect ratio. This can be changed with `preserveAspectRatio="none"`.

We have now covered all SVG tags and attributes you will most likely use with D3. The remaining sections will explore a few other tags and attributes that you might not use as frequently.

Other SVG elements

This section covers SVG elements for reusable symbols and markers, gradients, clippings, masks, filters, and animation. Most provide functionality that may be easier to implement in CSS (e.g., filters, masks, and gradients) or D3 (e.g., animations). In some cases, however, they may be a simpler alternative.

Symbols

Reusable shapes can be created with `<symbol>`. They are more flexible than using `<g>` for this purpose since they can be configured with `viewBox`:

```
<svg width="460" height="300" >
  <defs>
    <symbol viewBox="-15 -15 30 30" width="30" height="30" id="earth">
      <circle r="12" fill="none" stroke="currentColor"></circle>
      <rect x="-12" y="-1" height="2" width="24" fill="currentColor"/>
      <rect y="-12" x="-1" width="2" height="24" fill="currentColor"/>
    </symbol>
  </defs>
  <use href="#earth"/>
  <use href="#earth" transform="scale(10)" color="green"/>
  <use href="#earth" transform="translate(300,75) scale(5)" color="orange"/>
</svg>
```

The preceding code produces the result shown in *Figure 3.26*.

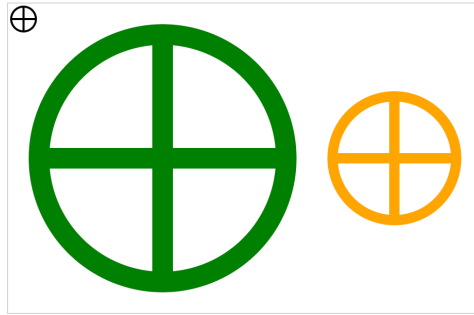


Figure 3.26 – A symbol created with a nested `<svg>` or `<symbol>` container. Code: `Other/SVG/3-symbols.html`

The same result is obtained with a nested `<svg>` (see `Containers/SVG/5-svg-reuse.html`), but symbols are more efficient, as they are only created when referenced by a `<use/>` element, if necessary.

Markers

A marker is a reusable symbol that can be attached to line ends and vertices. It is commonly used for arrowheads or to highlight points in a shape. To create one, declare the `<marker>` tag in a `<defs>` block, determine its user space (`viewBox`), and add shapes that fit in that space. To use a marker, you need to link it to a shape. The size of the marker will change proportionally to the shape's `stroke-width` property. *Figure 3.27* shows an example of a marker created in a 10x6 space.

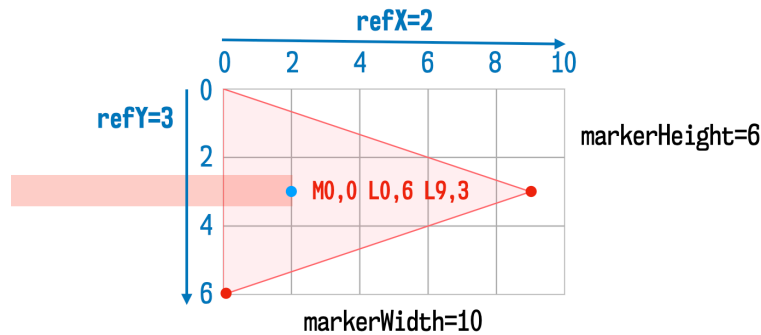


Figure 3.27 – A reusable SVG marker. Code: Other/SVG/4-marker-template.html

The following code defines the marker represented in *Figure 3.27*. The `orient="auto"` attribute guarantees that it will always be parallel to the line, attached at `refY` and `refX`:

```
<defs>
  <marker id="arrow" markerWidth="10" markerHeight="6"
    refY="3" refX="2" orient="auto">
    <path d="M0,0 L0,6 L9,3 z" fill="red"/>
  </marker>
</defs>
```

A marker can be attached to a shape using `marker-mid`, to apply it to all vertices; `marker-end` to apply it to the end of a line; or `marker-start`, to apply it to the beginning of the line. The same marker can be reused at the beginning, rotated 180 degrees if `orient` is defined as `auto-start-reverse`. Note that the link requires `url(#id)` (instead of simply `#id`, used in `href` attributes):

```
<path d="..." fill="none" stroke="red" stroke-width="4" marker-end="url(#arrow)" .../>
```

The line in *Figure 3.28* applies the marker to a path using `orient=auto-start-reverse`. It also applies composite markers to the vertices. See the complete code in `Other/SVG/5-marker.html`.

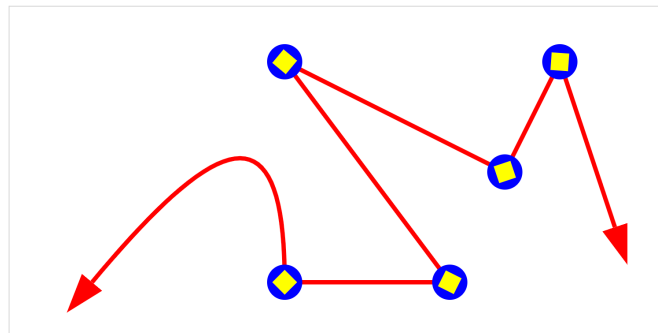


Figure 3.28 – Using markers applied to a path. Code: Other/SVG/5-marker.html

It's usually simpler to apply clipping, masks, and filters via CSS, when using SVG in HTML, but a library such as D3 can give you more control over these effects when implemented in SVG. Let's see how to use them.

Clipping paths

A clipping path allows you to clip off parts of any graphics. It is typically used with images. To create one, add a `<clipPath>` element to the `<defs>` header containing or referencing the shape that will provide the mask and declare an `id` value so it can be referenced by the object to be clipped:

```
<defs>
  <clipPath id="clip">
    <circle r="139" cx="200" cy="199"/>
  </clipPath>
</defs>
```

To use a clipping path, reference it with `url(#id)` inside a `clip-path` attribute:

```
<image x="25" y="25" height="350" width="350" href="../../data/images/pluto.jpg"
  clip-path=url(#clip)/>
```

The preceding code defines a circle as the clipping mask. When composed with an image, only the parts of the image that fit inside the circle will be preserved. The rest will be cropped. *Figure 3.29* shows the original image (left) and the clipped image (right).

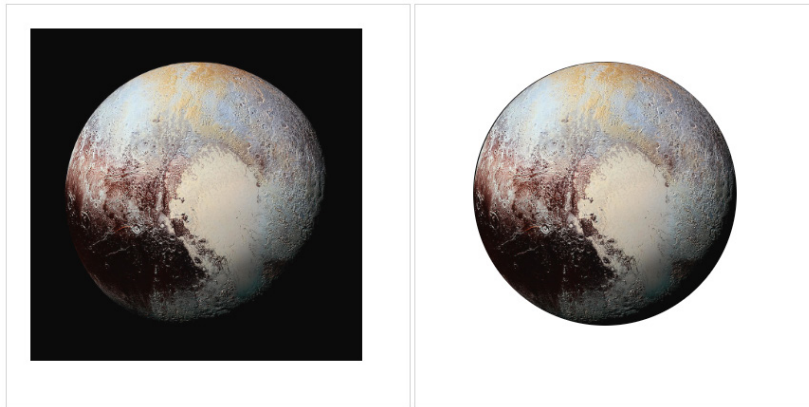


Figure 3.29 – Clipping or masking an image in SVG. Code: `Other/SVG/6-clipping.html` or `7-mask.html`

Clipping is great if you need to apply an all-or-nothing effect on an image. If instead you wish to fade parts of the image or apply a grayscale filter, you can use `<mask>`.

Masks

The `<mask>` element uses the luminance of a color as a level of opacity when applied to an object. This affects how the object interacts with anything *behind* it. White is opaque, and black is transparent. The following code uses `<mask>` for the same effect as `<clipPath>` (see `Other/SVG/7-mask.html`):

```
<defs>
  <mask id="mask">
    <rect x="25" y="25" height="350" width="350" fill="white"/>    <!-- Opaque -->
```

```

    <circle r="139" cx="200" cy="199" fill="black"/>      <!-- Transparent -->
  </mask>
</defs>

```

Here, the mask is applied to a white square placed on the image. It will cut open a transparent circle on the square, revealing the image behind it:

```

<image x="25" y="25" height="350" width="350" href="../../data/images/pluto.jpg"/>
<rect x="25" y="25" height="350" width="350" fill="white" mask="url(#mask)" />

```

The result should be identical to *Figure 3.29*. But with masks, you can apply tones of gray to create semitransparent masks. If you use colors, their luminance is applied to the filter. Images can be used to configure the mask (see [Other/SVG/8-image-mask.html](#)), but to compose in color, you need a filter.

Filters

SVG filters can be applied to shapes, text, or images. They are usually created in the `<defs>` header and referenced via `url(#id)` using the `filter` attribute. The `<filter>` element creates a filter by configuring a filter type (elements that start in `<fe*>`). You can create composite filters by adding more filter types.

This `<defs>` block defines two configurations for the `<feGaussianBlur>` filter:

```

<defs>
  <filter id="filter1">
    <feGaussianBlur stdDeviation="7" />
  </filter>
  <filter id="filter2" x="-100" y="-100" height="200" width="200">
    <feGaussianBlur stdDeviation="0,5" in="SourceGraphic" />
  </filter>
</defs>

```

To use the filters, reference them from a shape via the `filter` attribute, as shown in the following code block. This code will apply the second filter to the text, and the first one to the blue circle:

```

<text id="text" x="50" y="60" filter="url(#filter2)">Do you need glasses?</text>

<g id="stardot" transform="translate(100,25) scale(0.5)">
  <polygon points="250,0 400,500 0,200 500,200 100,500"
    fill="red" fill-rule="evenodd"/>
  <circle cx="250" cy="283" r="75" fill="blue" filter="url(#filter1)" />
</g>

```

The result is shown in *Figure 3.30*.



Figure 3.30 – A Gaussian blur filter applied to different shapes. Code: Other/SVG/9-filter.html

These are just a few of the many filters and effects that you can apply to your shapes using SVG.

Gradients

SVG supports two types of gradients: linear and radial. Linear gradients are perpendicular to a line and require the same attributes as the `<line>` element. Radial gradients use `<circle>` attributes.

Gradients typically declare two or more child `<stop>` colors, which configure each color with an offset and stop-color. The following `<defs>` block declares two gradients:

```
<defs>
  <linearGradient x2="1" id="rainbow">
    <stop offset="0" stop-color="rgb(255,0,0)" />
    <stop offset="0.25" stop-color="rgb(255,255,64)" />
    <stop offset="0.5" stop-color="rgb(64,255,64)" />
    <stop offset="0.75" stop-color="rgb(64,64,255)" />
    <stop offset="1" stop-color="rgb(128,0,255)" />
  </linearGradient>
  <radialGradient cx="0.35" cy="0.35" r="1" id="glow">
    <stop offset="0" stop-color="rgb(255,255,255)" />
    <stop offset="0.5" stop-color="rgb(0,128,255)" />
    <stop offset="1" stop-color="rgb(128,0,255)" />
  </radialGradient>
</defs>
```

After you declare it, you can use a gradient as a color and reference it in the fill and stroke attributes via `url(#id)`. Here, they are used to stroke and fill three shapes:

```
<rect width="300" height="50" fill="url(#rainbow)" transform="translate(50,25)"/>
<rect width="300" height="50" stroke="url(#rainbow)" fill="none" stroke-width="10"
  transform="rotate(180,200,145) translate(50,120)"/>
<circle r="75" fill="url(#glow)" transform="translate(475,100)"/>
```

The results are shown in *Figure 3.31*.

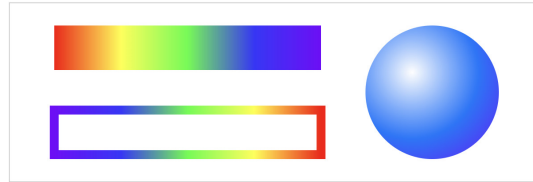


Figure 3.31 – Gradients applied to rectangles (linear) and a circle (radial). Code: Other/SVG/10-gradient.html

You can use gradients to display different colors in a heatmap.

Patterns

Repeatable patterns can be created with the `<pattern>` element, which provides a local coordinate system for the shapes that will repeat. The following code creates a simple crosshatch pattern:

```
<pattern id="crosshatch" viewBox="0 0 10 10" width="10" height="10"
  patternUnits="userSpaceOnUse">
  <path d="M0,0 L0,2 8,10 10,10 z"/>
  <path d="M8,0 L10,0 10,2 z"/>
  <path d="M0,0 L0,2 8,10 10,10 z" transform="rotate(90,5,5)"/>
  <path d="M8,0 L10,0 10,2 z" transform="rotate(90,5,5)"/>
</pattern>
```

The `patternUnits` attribute is used to preserve the size of the shapes when they are rendered. Once declared, the pattern can be used as a fill or stroke color. Here, it's used to paint a rectangle:

```
<rect height="50" width="100" fill="url(#crosshatch)" transform="translate(50,120)" />
```

The preceding code was used to paint the third rectangle in *Figure 3.32*.

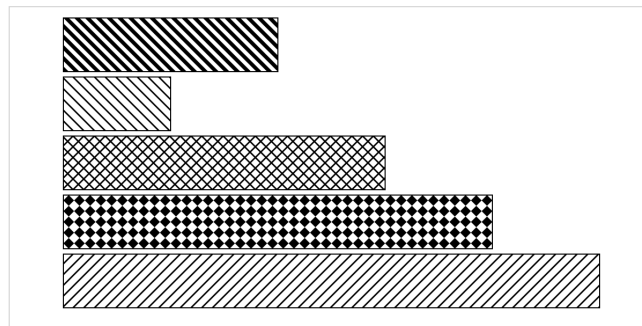


Figure 3.32 – A pattern used as a fill color and as a stroke color. Code: Other/SVG/11-pattern.html

The preceding patterns were created in black, but they can be any color. They can even include images. Patterns are a great alternative to paint charts with textures when you can't use colors.

Animation elements

SVG supports animation using the `<animate>`, `<animateMotion>`, `<animateTransform>`, and `<set>` tags. Much of this functionality is duplicated in other widely available APIs, such as CSS animations and transitions. D3 also provides its own API for animated transitions, which is easier to use for data visualizations.

Some animations may be simpler to implement in SVG, such as moving an object along a path. We will explore them in *Chapter 13*. See some animations created with these tags in the `Animation/` folder.

The final section in this chapter explores advanced concepts that involve JavaScript programming and the SVG DOM. If you wish, you can skip it and refer to it later when you need it.

Using the DOM API

SVG elements can be selected using DOM methods such as `getElementById()` or `querySelector()`. While D3 eliminates most of the work in selecting and transforming elements, you might still need to perform operations on SVG elements. *Table 3.6* lists some useful methods you can use.

Method	DOM Interface	Description
<code>getCTM()</code>	<code>SVGGraphicsElement</code>	Returns the CTM (an <code>SVGMatrix</code> object).
<code>getScreenCTM()</code>	<code>SVGGraphicsElement</code>	Returns the CTM (<code>SVGMatrix</code>) for a graphics element in screen coordinates (relative to page, not SVG viewport).
<code>getTotalLength()</code>	<code>SVGGeometryElement</code>	Returns the total length of a path, line, polyline, or other geometry element.
<code>getPointAtLength(<i>dist</i>)</code>	<code>SVGGeometryElement</code>	Returns an <code>SVGPoint</code> at a certain <i>distance</i> from the start of a path.
<code>isPointInFill</code>	<code>SVGGeometryElement</code>	Returns <i>true</i> if a point (must be an <code>SVGPoint</code> object) is in the fill area of an object.
<code>isPointInStroke</code>	<code>SVGGeometryElement</code>	Returns <i>true</i> if a point (must be an <code>SVGPoint</code> object) is in the stroke area of an object.
<code>getComputedTextLength()</code>	<code>SVGTextContentElement</code>	Returns the length of rendered text.
<code>getBBox</code>	<code>SVGGraphicsElement</code>	Returns the bounding rectangle of a shape (an <code>SVGRect</code>). Does not consider any transforms applied to the shape. Coordinates are relative to the SVG viewport.
<code>getBoundingClientRect()</code>	<code>Element</code>	Gets the bounding rectangle of a shape (a <code>DOMRect</code>). Coordinates are relative to the page.
<code>elementsFromPoint()</code>	<code>Document</code>	Returns an array of elements that contain a given point.

Table 3.6 – Some useful DOM methods for SVG graphics

The methods in *Table 3.6* return three types of DOM interfaces: *rectangles* (DOMRect and SVGRect), with x, y, height, and width properties; *points* (DOMPoint and SVGPoint), with x and y properties; and *matrices* (DOMMatrix and SVGMatrix), with a, b, c, d, e, and f properties. Although they are considered aliases, they unfortunately can't yet be used interchangeably in all browsers. But DOM objects can be created from SVG objects with DOMPoint.fromPoint(), DOMRect.fromRect(), and DOMMatrix.fromMatrix().

Methods defined in SVG interfaces usually provide data relative to the SVG container, while other DOM interfaces based on the page or screen need to be adjusted to match the viewport's coordinate system.

Let's see some examples using these methods.

Using the current transformation matrix

The **CTM** stores information about cumulative transforms applied to an object in the coordinate system of its parent container. Calling getCTM() on a shape or group returns an SVGMatrix from where you can get its components (a, b, c, d, e, and f properties) and use them to create new transforms or apply an inverse transform on another object. For example, consider the following <rect>:

```
<rect ... transform="translate(45,80) rotate(-40) scale(1.5)" />
```

The following code (API/SVG/1-ctm.html) will obtain its CTM:

```
const rect = document.querySelector("rect");
const ctm = rect.getCTM();           // This is an SVGMatrix
```

Then use the CTM to apply the same transform to other objects, for example, to an existing path:

```
path.setAttribute("transform", `matrix(${ctm.a,ctm.b,ctm.c,ctm.d,ctm.e,ctm.f})`);
```

The DOMMatrix interface has a toString() method that generates the matrix() string required by transform. To use it, you must first convert the SVGMatrix into a DOMMatrix, as shown here:

```
const domMatrix = DOMMatrix.fromMatrix(ctm);
path.setAttribute("transform", domMatrix.toString());
```

The CTM contains methods, such as inverse(), which inverts the matrix. This is useful to obtain coordinates before a transformation (or viewBox configuration) or to simply revert a transform:

```
const itm = ctm.inverse();
```

Here, the inverse was used to undo a transform applied to a rectangle (API/SVG/2-ctm-inverse.html):

```
const inv = DOMMatrix.fromMatrix(itm);           // convert to DOMMatrix
rect.setAttribute("transform", inv.toString()); // generate matrix(a,b,c,d,e,f)
```

If you are working with operations that depend on the screen's coordinate system (e.g., mouse events), you can call `getScreenCTM()` to get the CTM relative to the page. For example, the event object received by an event handler contains `x` and `y` in *screen* coordinates. The following code uses the inverse screen CTM to convert them to *local* SVG coordinates so that the circle is placed in the correct position:

```
svg.addEventListener("click", event => {
  const stm = svg.getScreenCTM();
  const pt = DOMPoint.fromPoint(event).matrixTransform(stm.inverse());
  circle.setAttribute("cx", pt.x); // corrected coordinates
  circle.setAttribute("cy", pt.y);
})
```

The `fromPoint()` method accepts any object that has `x` and `y` properties (such as `event`), and `matrixTransform()` applies a matrix transform to the point (in this case, the inverse screen CTM).

Correcting individual points only fixes the offset. If your container has a `viewBox` or transforms such as scaling or rotation, you may need to apply the entire inverse transform to the circle itself:

```
const itm = DOMMatrix.fromMatrix(stm.inverse());
circle.setAttribute("transform", itm.toString()); // generates matrix(a,b,c,d,e,f)
```

See an interactive example using this code in [API/SVG/3-screen-ctm.html](#).

Getting lengths and bounds

To compute the total length of a line or path or the perimeter of any shape, call `getTotalLength()`. For example, this will return the circumference of a circle (see [API/SVG/4-lengths.html](#)):

```
const circle = document.querySelector("circle"); // any <circle>
console.log(`Circumference: ${circle.getTotalLength()} pixels`);
```

To fit a string of text in a rectangle, you must know its length after the string is rendered. You can get it by calling `getComputedTextLength()` on a `<text>` or `<tspan>` object:

```
const text = document.querySelector("text"); // <text>Lorem ipsum</text>
console.log(`Computed text length: ${text.getComputedTextLength()} pixels`);
```

If the text is rotated, has a curved baseline, or you want to know its center, you need its *bounding box*: a `DOMRect` or `SVGRect` that describes the smallest rectangle containing a shape or a group of shapes. *Figure 3.33* shows the bounding boxes of several shapes and groups:

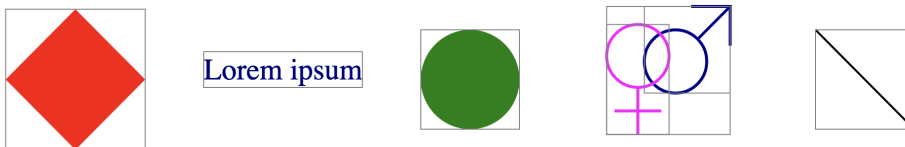


Figure 3.33 – Bounding boxes. Code: [API/SVG/5-bounding-client-rect.html](#) or [6-bbox.html](#)

There are two methods you can use to obtain a bounding box. Although they return different interfaces, they support the same *x*, *y*, *height*, and *width* properties that draw a rectangle. Both, however, may require some corrections. For example, consider the circle in *Figure 3.33* defined by the following SVG:

```
<circle r="50" transform="translate(500,100)" fill="green"/>
```

Calling `getBoundingClientRect()` returns a `DOMRect` object. Since it is not an SVG object, *x* and *y* are returned in screen coordinates and need to be converted to local coordinates, applying the inverse CTM transform to the points, as shown here:

```
const box = circle.getBoundingClientRect(); // {height:100, width:100, x:459, y:287}
const stm = svg.getScreenCTM();           // {a: 1, b: 0, c: 0, d: 1, e: 9, f: 237}
const pt = DOMPoint.fromPoint(box).matrixTransform(stm.inverse());
box.x = pt.x; // x: 450
box.y = pt.y; // y: 50
```

Since `getBBox` is SVG-specific, it returns an `SVGBBox` object in local coordinates, but unfortunately, it doesn't consider transforms. The bounding box for the circle treats it as if it were at the origin:

```
const box = circle.getBBox(); // {height: 100, width: 100, x: -50, y: -50}
```

The transform is a simple translation, which can be fixed using the circle's local CTM:

```
const ctm = circle.getCTM(); // {a: 1, b: 0, c: 0, d: 1, e: 500, f: 100}
box.x = ctm.e; // x: 450
box.y = ctm.f; // y: 50
```

But this will be much harder if you have scaled units (`viewBox`), rotations, and cumulative transforms. For example, the rectangle in *Figure 3.33* is rotated 45 degrees:

```
<rect x="50" y="50" width="100" height="100" fill="red"
      transform-origin="100 100" transform="rotate(45)"/>
```

You can't rotate a bounding box. A solution is to place the rotated rectangle inside a `<g>` container, translate the container (if necessary), and get the container's bounding box. In such cases, it's probably simpler to use `getBoundingClientRect()` and compensate the screen offset.

Finding points

Calling `getPointAtLength(distance)` on a path returns the point at the specified position. This method can be used to place an object on the border of a shape. If you animate the distance, you can use it to make an object move along a path (see [API/SVG/7-point-at-length.html](#)).

Although `getPointAtLength()` is an SVG method, it doesn't consider transforms. Consider the following setup, with a scaling `viewBox` applied to the viewport, and a vertical scale applied to the path:

```
<svg width="1000" height="500" viewBox="0 0 500 250">
  <path d="..." ... transform="scale(1,.5)"/>
</svg>
```

To obtain the correct coordinates for the points, you need to invert their `viewBox` transforms, and then apply the path transforms, as follows (see [API/SVG/8-point-transforms.html](#)):

```
const pt = path.getPointAtLength(distance); // point not considering transforms
const spt = pt.matrixTransform(svg.getCTM().inverse()); // invert SVG viewBox
const point = spt.matrixTransform(path.getCTM()); // apply path transform
```

The best way to discover whether a point is in a shape is with `elementsFromPoint()`, a DOM method that returns an array with all elements containing the point. It uses screen coordinates (it's not an SVG method). For example, this prints all elements that contain a clicked point ([API/SVG/9-elements-from-point.html](#)):

```
svg.addEventListener("click", function(e) {
  const pt = DOMPoint.fromPoint(e);
  const elements = document.elementsFromPoint(pt.x, pt.y);
  console.log(elements);
});
```

You can use `isPointInFill()` and `isPointInStroke()` to test whether a point is in a shape's fill or stroke (see [API/SVG/10-in-fill-stroke.html](#)). They work in simple scenarios but may require complex code if you must consider transforms and `viewBox`. Using `elementsFromPoint()` is usually simpler in these cases.

These methods are used in [API/SVG/11-locating-points.html](#) (*Figure 3.34*), where the color of each circle is assigned based on its position in relation to the shape. You can also click anywhere on the SVG to draw a square of the corresponding color.

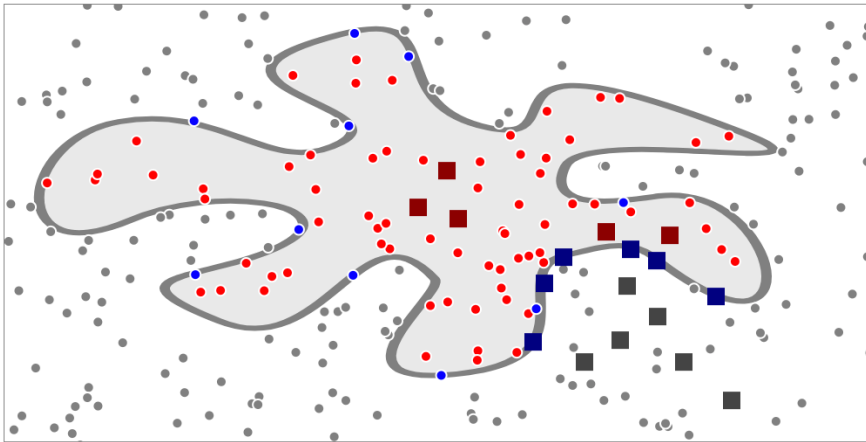


Figure 3.34 – Discovering whether a point is inside a shape or on its border. Code: [API/SVG/11-locating-points.html](#)

This section covered several advanced concepts using the SVG DOM. The code is in the `API` folder. The `SVG/` subfolder contains examples in pure JavaScript, while the `D3/` subfolder implements them using D3.

Before moving on to the next chapter, open a web page and try out some examples from this chapter; create shapes and apply fills, stroke effects, and filters. Try drawing some polygons and simple paths and experiment with applying transforms to groups of elements. This will make your D3 journey much smoother.

Summary

This chapter provided a detailed overview covering essential topics in the SVG technology, the main open standard that D3 uses to draw graphics. Advanced topics, such as matrix transforms and DOM methods, were also covered.

Although this chapter explored the most important SVG elements and attributes, many other features were left out. See *Chapter 20* for links to specs and documentation about SVG.

In the next chapter, you will finally dive into the world of D3 and create a complete interactive animated bar chart using SVG shapes, exploring a little of the features we will cover in detail later in this book.

Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.

Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.

UNLOCK NOW



